# SketchRAG: Approximate Streaming Summaries as Retrieval Context for Agentic AI

R Ranjith Kumar
2023BCD0030
IIIT Kottayam

Bhuvan S M
2023BCS0198
IIIT Kottayam

Hariprashanth M
2023BCS0036
IIIT Kottayam

Nooh K
2023BCD0003
IIIT Kottayam

***Abstract*—Agents that read continuous streams cannot afford to keep and search every chunk forever. We present SketchRAG, which maintains a bounded, query-independent summary of a text stream—semantic cells with extractive digests, per-cell Count-Min Sketches, Misra–Gries heavy hitters and streaming IDF, all updated in one pass—and uses it as the retrieval layer. Its key component is an *error-bound gate*: the Count-Min guarantee, checked against each cell's digest, decides when the summary cannot be trusted and raw evidence must be read from cold storage. On 350 LongBench and $\infty$-Bench questions answered by qwen2.5:7b-instruct, the gated system scores 41.7, ahead of full context (38.1), dense RAG (35.8), SARA-lite (38.2) and AMBER-lite (32.0), while sending the fewest LLM context tokens of any system (1,213 per question, 8.5% of full context). BM25 RAG remains the strongest system (46.0). The gate is calibrated: on the 54% of questions it escalates, raw evidence lifts the score from 30.0 to 51.5, while on the questions it trusts, always reading raw text would add under one point. With a shared base model, SketchRAG matches the official EM-LLM implementation (34.8 vs. 34.5). On an 8.5M-word stream, hot memory stays at 1.0–3.7 MB against a 146 MB dense index, but answer recall falls (0.23 vs. 0.44), which is the main limitation we report.**

***Index Terms*—retrieval-augmented generation, data streams, Count-Min Sketch, LLM agents, long context, memory**

## I. Introduction

Large language model (LLM) agents increasingly work over information that arrives continuously: logs, transcripts, reports and documents appended over time. Two families of methods give such an agent access to its history. Retrieval-augmented generation (RAG) [1] keeps every chunk and a vector for each one, so its searchable index grows linearly with the stream. Long-context and episodic-memory methods such as EM-LLM [2] avoid retrieval over text but keep per-event key/value state for the whole history and need access to model internals. Simply placing everything in the prompt is costly and brittle: models lose information in the middle of long inputs [3], and documents longer than the window must be truncated.

The data-stream literature offers a different tool: *synopses* that summarise an unbounded stream in bounded memory with provable error [4]. This project asks whether such synopses can serve as the retrieval layer of an agent. Our central research question is: *can approximate streaming summaries provide a memory-efficient retrieval context for agentic AI while keeping answer quality comparable to full-context and RAG approaches?* We study it through five sub-questions: whether the summaries preserve enough information (RQ1), how the approach compares with episodic-memory, adaptive-memory and compression methods (RQ2), how the memory budget affects accuracy (RQ3), whether the system can decide when the summary is insufficient (RQ4), and which trade-off between memory, compression, latency and accuracy is best (RQ5).

**Contributions.** (i) SketchRAG, a single-pass streaming memory that combines streaming clustering, per-cell Count-Min Sketches [5], Misra–Gries heavy hitters [6] and extractive digests, maintained *before* any query and bounded in size regardless of stream length. (ii) An error-bound gate that turns the Count-Min guarantee into a per-cell, per-query signal for whether the summary can be trusted, with raw evidence fetched from cold storage only for escalated cells. (iii) An evaluation on 350 test questions over 12 tasks against full context, BM25 and dense RAG, re-implementations of SARA [7] and AMBER [8], a reproduction of EM-LLM from its official code, ablations, memory-budget sweeps and an 8.5M-word stress test, all run with open models served by Ollama [9].

## II. Related Work

**Episodic memory.** EM-LLM [2] segments the token stream into events at points of high surprise, stores each event's key/value cache, and at inference retrieves similar events plus temporally contiguous neighbours. It scales to very long inputs without fine-tuning, but its memory grows with the input and it needs the model's internals and a large GPU. Our cells play the role of events and our temporal-neighbour rule follows its contiguity buffer, but we operate on text and keep a fixed budget.

**Adaptive and agentic memory.** AMBER [8] builds a memory note during each question with several LLM roles: a memory updater, an adaptive collector that decides whether to retrieve again, and a multi-granular filter. MemGPT [10] lets the LLM page information in and out of its context. Both put the LLM in charge of memory decisions, which costs extra calls; we test this choice directly (an LLM-driven variant of SketchRAG) and find that a deterministic sketch signal is better calibrated.

**Context compression.** SARA [7] retrieves generously and compresses the retrieved evidence to a fixed budget, mixing natural-language snippets with learned compression vectors. Compression happens *after* a query. SketchRAG compresses the *stream* continuously, before any query arrives.

**Streaming synopses.** The Count-Min Sketch [5] estimates item frequencies in $O(\frac{1}{\varepsilon}\log\frac{1}{\delta})$ space, never under-counting and over-counting by at most $\varepsilon N$ with probability $1-\delta$. Misra–Gries [6] keeps every item with frequency above $N/(k{+}1)$ using $k$ counters, and reservoir sampling [11] keeps a uniform fixed-size sample. Clustering features [12] and merge-and-reduce [13] cluster streams in bounded memory. To our knowledge, these synopses have not been used as the retrieval layer of an LLM agent, nor has a sketch's error bound been used to decide when to read raw evidence.

## III. Preliminaries: Streaming Synopses

We use the cash-register stream model [4]: items (here, terms of arriving chunks) arrive one at a time, each update must be processed in small time and space, and $N$ denotes the total count seen so far.

**Count-Min Sketch.** A Count-Min Sketch [5] is a $d \times w$ table of counters with $d$ pairwise-independent hash functions $h_1, \ldots, h_d$. An update of item $x$ increments $C[j, h_j(x)]$ for every row $j$, and a query returns $\hat{f}(x) = \min_j C[j, h_j(x)]$. With $w = \lceil e/\varepsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$,

$$f(x) \;\le\; \hat{f}(x) \;\le\; f(x){+}\varepsilon N \quad \text{with probability } \ge 1{-}\delta. \tag{1}$$

Our cells use $w{=}512$ and $d{=}3$, so $\varepsilon \approx 0.0053$ and $\delta \approx 0.05$. We use the *conservative update*, which raises only the counters that equal the current minimum; it keeps the guarantee and reduces over-counting. The sketch is linear: the table of a union of two streams is the sum of their tables. The gate uses two thresholds derived from it: the expected collision mass of one row, $\nu = N/w$ (a tight but non-guaranteed noise floor), and the guaranteed bound $\varepsilon N$ of (1).

**Misra–Gries.** With $k$ counters, Misra–Gries [6] keeps a candidate set of heavy items; when a new item finds no free counter, all counters are decremented. Every item with frequency above $N/(k{+}1)$ survives, and each estimate is at most $N/(k{+}1)$ below the truth. We use $k{=}256$ to expose the stream's topics.

**Streaming IDF.** A second, global Count-Min Sketch counts the number of chunks containing each term, giving a document frequency $\widehat{df}(t)$ and $\mathrm{idf}(t) = \log\frac{n+1}{\widehat{df}(t)+1} + 1$ after $n$ chunks. Because the sketch over-counts, the IDF it yields is a conservative (low) estimate, so a term that looks rare really is rare.

## IV. Method

Fig. 1 gives an overview. A document or stream is split into chunks of about 150 word units (long machine tokens such as UUIDs are charged per six characters so that a chunk holds a similar number of LLM tokens). Each chunk is embedded once with bge-small-en-v1.5 [14] and passed to the memory, which updates two stores.

### A. Hot memory

The hot memory is the only part searched at query time and its size is fixed by the configuration.

TABLE I
Hot-memory accounting, measured on a 194-chunk NarrativeQA stream with $K{=}64$ cells (total 762 KB). Cold storage is not included.

| Component | Bytes | Scope |
|---|---|---|
| Centroid sum (384 × fp32) | 1,536 | per cell |
| Count-Min Sketch (3 × 512 int32) | 6,144 | per cell |
| Exemplars (≤8 × 384 fp16; mean here) | 2,244 | per cell |
| Extractive digest (≤120 words; mean) | 669 | per cell |
| Document-frequency CMS (3 × 8192) | 98,304 | global |
| Misra–Gries table ($k{=}256$) | 2,492 | global |

**Semantic cells.** Each cell is a clustering feature [12]: the sum $\mathbf{s}$ and count $n$ of its member embeddings, from which the centroid $\mathbf{s}/\|\mathbf{s}\|$ and radius follow (for unit vectors $\mathbb{E}\|x-\mu\|^2 = 1-\|\mu\|^2$, $\mu = \mathbf{s}/n$), plus the span of stream positions it covers. A chunk joins the most similar cell if cosine similarity is at least 0.90 (with a small bonus for the current cell, following EM-LLM's event continuity); otherwise it opens a new cell. When the number of cells exceeds the budget $K$, the pair with the smallest Ward cost [15] $\frac{n_a n_b}{n_a+n_b}\|\mu_a-\mu_b\|^2$ is merged (merge-and-reduce [13]). Ward linkage resists a single cell absorbing the whole stream.

**Per-cell Count-Min Sketch.** Each cell counts its content terms in a $3 \times 512$ sketch with conservative update. Because the sketch is linear, merging two cells adds their tables, so the guarantee survives merges.

**Digest and exemplars.** Each cell keeps an extractive digest of at most 120 words: the most salient sentences, scored by the mean streaming IDF of their terms and by how typical their chunk is of the cell. No LLM is called at ingest time. A cell also keeps up to eight diverse member embeddings (exemplars), selected by repeatedly dropping the most redundant one; they play the role of EM-LLM's representative tokens when cells are ranked.

**Global synopses.** A Count-Min Sketch of document frequencies gives a streaming IDF, and a Misra–Gries table of the 256 heaviest terms gives the stream's topics, shown to the LLM as global context.

### B. Memory and time complexity

Table I accounts for the hot memory. A cell occupies 10.4 KB on average and at most 14.2 KB, dominated by its sketch and exemplars, plus about 100 KB of global synopses; a 64-cell memory is therefore about 1 MB however long the stream is. Ingesting a chunk costs $O(Kd)$ to score the $K$ cells against the $d{=}384$-dimensional embedding, $O(L)$ sketch updates for its $L$ terms, and, when the budget is exceeded, an $O(K^2 d)$ Ward merge (we recompute the pairwise cost matrix; at $K \le 256$ this takes milliseconds). Answering costs $O(KEd)$ to rank cells with $E \le 8$ exemplars each, $O(|R|)$ sketch queries per retrieved cell for the gate, and, for escalated cells only, a scan of that cell's pointer list in cold storage.

### C. Cold storage

Every raw chunk, its embedding and the cell→chunk pointer lists are appended to cold storage, which is never searched and

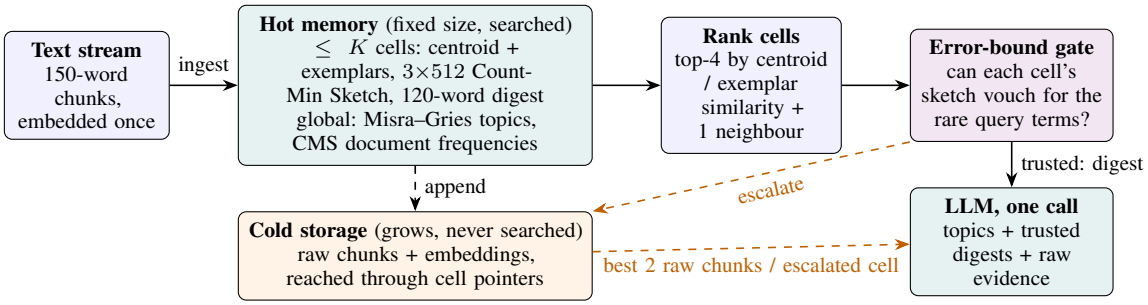



Fig. 1. SketchRAG. Each chunk updates a fixed-size hot memory in one pass and is appended to cold storage. At query time cells are ranked, and the gate escalates only the cells whose Count-Min Sketch cannot vouch for their digest; their raw chunks replace the digest in the LLM's context.

**Algorithm 1** Error-bound gate for one query

**Input:** retrieved cells $C$, rare query terms $R$
**Output:** cells $E \subseteq C$ escalated to raw evidence

$E \leftarrow \emptyset$; *conf* $\leftarrow$ **false**
**for** $c \in C$ **do**
  **if** $\mathrm{radius}(c) > 0.75$ **then** ▷ *loose*
    $E \leftarrow E \cup \{c\}$
  **end if**
  **for** $t \in R$ **do**
    $\hat{f} \leftarrow \mathrm{CMS}_c(t)$; $\nu \leftarrow N_c/w$
    **if** $\hat{f} > \varepsilon N_c$ **then** *conf* $\leftarrow$ **true**
    **end if**
    **if** $\hat{f} - \nu \geq 1 \wedge t \notin \mathrm{dig}(c)$ **then** ▷ *lossy*
      $E \leftarrow E \cup \{c\}$
    **end if**
  **end for**
**end for**
**if** $R \neq \emptyset \wedge \neg$*conf* **then** ▷ *unresolved*
  $E \leftarrow E \cup \mathrm{top}_2(C)$
**end if**
**return** $E$

is read only when the gate escalates a cell. Cold storage grows with the stream; we report hot and cold sizes separately and never describe the system as bounded end to end.

### D. Query time and the error-bound gate

A question is embedded and each cell is scored by the maximum cosine similarity over its centroid and exemplars. The top four cells are kept, plus the temporal neighbour of the best cell (EM-LLM's contiguity). The gate (Algorithm 1) then examines the *rare* query terms, those with streaming IDF at least 2.5.

A cell is escalated when it is *lossy* (the sketch places a rare term in the cell above its noise floor $N/w$, but the digest lacks it, so the summary dropped the fact; we also require some digest sentence to hold two rare terms together), *loose* (its radius exceeds 0.75, so its digest averages too much), or *unresolved* (no retrieved cell holds any rare term above the guaranteed bound $\varepsilon N$, so the sketch cannot locate the answer and the top two cells are read). Questions without rare terms, such as summarisation requests, are answered from the digests. For each escalated cell, its cold chunks are re-ranked by cosine similarity plus IDF-weighted term overlap and the best two replace the cell's digest. The LLM is called once with the heavy-hitter topics, the trusted digests and the raw evidence.

**Policies.** We compare five SketchRAG policies: SKETCH (digests only), RAW (escalate every retrieved cell), GATE (Algorithm 1), AGENT (the LLM sees the digests and may call `fetch_raw`/`search_memory` tools for up to three rounds) and HYBRID (the gated context plus the same tools).

## V. Implementation

SketchRAG is implemented from scratch in Python and NumPy: the Count-Min Sketch, Misra–Gries, reservoir sampling, streaming clustering and the gate are our own code, because SciPy and scikit-learn were blocked on our development machine. Unit tests check on synthetic Zipf streams that the Count-Min bound (1) holds, survives merging and is tightened by conservative update, that Misra–Gries keeps its guarantee, that reservoir samples are uniform, and that hot memory stays bounded while cold storage grows; all 12 pass. Every system implements one interface that turns a question and its document stream into an LLM context, and a single runner calls the LLM with the same prompt for all of them. Chunk embeddings are computed once on a GPU and cached in a content-addressed SQLite store, so every system and every run sees identical vectors.

All LLM calls go through Ollama [9], which lets the same code run against a local CPU server during development and against a GPU server inside Kaggle notebooks for the bulk runs. The runner is resumable and writes one JSON record per (system, question), so a crashed notebook loses no finished questions; the full set of runs took about 6.4 GPU-hours for the main comparison and 1.3 for ablations and sweeps. Every table and figure in this report is regenerated from these records by a script, and every number in the text is a macro produced by the same script.

## VI. Experimental Setup

**Data.** The test split is a fixed-seed sample (seed 0) of 30 questions from each of nine English LongBench tasks [16] (NarrativeQA, Qasper, MultiFieldQA, HotpotQA, 2WikiMQA, MuSiQue, GovReport, QMSum, PassageRetrieval) and two $\infty$-Bench retrieval tasks [17] (PassKey, KV), plus 20 En.QA

questions over books of about 190K tokens: 350 questions in total. Each document is fed to every system as a chunk stream. All settings were tuned on a disjoint development split (seed 1000, 10 questions per task); the test split was frozen before tuning. The *stress test* concatenates every English LongBench QA and summary document (1,115 documents, 8,488,204 words, 65,127 chunks) into one stream in a fixed shuffled order and asks 100 probe questions whose source documents are spread evenly over it.

**LLM.** Every system uses qwen2.5:7b-instruct [18] served by Ollama on a Kaggle T4 GPU, with a 32,768-token window, greedy decoding, a fixed seed and the same prompt template; only the context differs. Full context is middle-truncated to the window following LongBench (105 of its 350 prompts needed truncation).

**Baselines.** *Full context*; *BM25 RAG* [19] (top-5 chunks); *dense RAG* (FAISS [20] inner product over the same bge embeddings, top-5); *SARA-lite* (dense top-20, compressed to a 600-word budget by sentence selection; SARA's learned compression vectors are not reproduced); *AMBER-lite* (up to three rounds in which the LLM updates a memory note, decides whether to retrieve again and filters sentences; AMBER's trained filters are not reproduced); and *EM-LLM* from its official repository with its Phi-3.5-mini [21] configuration on two Kaggle T4 GPUs. For a like-for-like comparison, SketchRAG and the RAG baselines are rerun on the same question subset with the same base model (phi3.5 through Ollama).

**Metrics.** Answer quality uses LongBench's official metrics: token F1 for QA, ROUGE-L for summaries, accuracy for passage retrieval and exact containment for PassKey and KV; scores are averaged per task and then over tasks. Retrieval quality is *answer recall*, whether a gold answer string appears in the context sent to the LLM, a proxy because the benchmarks lack evidence annotations. Efficiency is measured as LLM context tokens and calls per question, LLM time, hot-memory bytes, ingest time per chunk and the gate's fallback rate.

## VII. Results

### A. Answer quality and cost (RQ1, RQ2, RQ5)

Table II gives per-task scores and Table III the costs. SketchRAG-gate scores 41.7, ahead of full context (38.1), SARA-lite (38.2), dense RAG (35.8) and AMBER-lite (32.0), while sending 1,213 context tokens per question, the fewest of any system and 8.5% of full context, with one LLM call and 5.8 s of LLM time against 61.5 s for full context (Fig. 2). Plain BM25 RAG is the strongest system (46.0), mainly through exact-match tasks: on PassageRetrieval it scores 70.0 against 53.3. Full context fails PassKey (16.7) because middle truncation removes the needle.

The summary alone is not enough (RQ1): SKETCH scores 29.9, losing most on questions that need an exact string; on $\infty$-Bench KV it scores 6.7 and the gate lifts this to 96.7. On global questions the summary suffices: GovReport has no rare query terms, the gate never fires (0%) and all three policies score 11.7–13.2. SketchRAG trails on multi-hop QA, where evidence is spread over more chunks than five cells hold: on 2WikiMQA the gate scores 33.3 against 51.9 (SARA-lite) and 58.7 (full context).

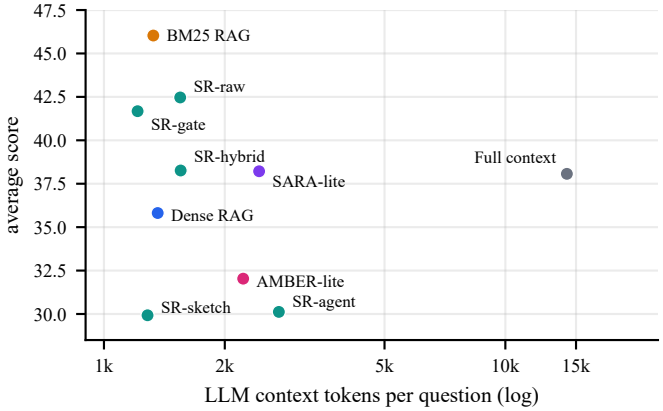



Fig. 2. Answer quality vs. LLM context tokens (log scale). SketchRAG-gate (SR-gate) has the cheapest context of any system and the third-best score.

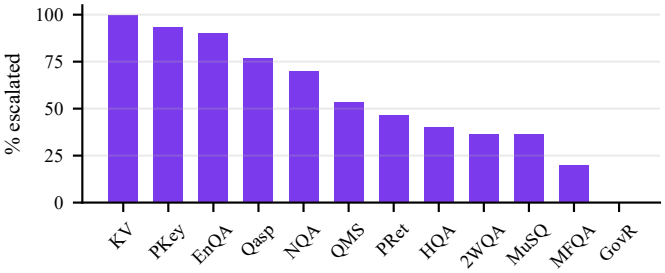



Fig. 3. Share of questions the gate escalates, per task. Exact-key tasks (KV, PassKey) almost always escalate; summarisation (GovR) never does.

### B. Is the gate calibrated? (RQ4)

The gate escalates 190 of the 350 questions (54%). On those, raw evidence lifts the score from 30.0 (digest only) to 51.5. On the 160 questions it trusts, always reading raw text would have moved the score only from 31.4 to 32.2. It fires on 100% of KV and 93% of PassKey questions, where digests drop exact keys, and never on GovReport. Overall it recovers 94% of the gain of always reading raw evidence while sending 22% fewer tokens than RAW. Fig. 3 shows the fire rate per task: it follows how much each task depends on exact strings.

Table IV shows real answers. In the NarrativeQA question the digest alone makes the model answer "not enough information"; the gate finds no cell that can vouch for the rare name, reads two cells raw, and the model names the victim, while full context, whose prompt was truncated, names the wrong baron. For the KV key no summary can hold a UUID, and only the raw chunk recovers the value. In the MuSiQue question the sketch places the rare terms in a cell whose digest dropped them (*lossy*), and the raw text supplies the answer. The 2WikiMQA row is a failure typical of multi-hop questions: the gate escalates, but the evidence for the second entity lies in cells that were never retrieved, so reading raw text cannot help.

TABLE II

ANSWER QUALITY ON THE 350 TEST QUESTIONS (TASK SCORE ×100; F1 FOR QA, ROUGE-L FOR GOVR/QMS, ACCURACY FOR PRET/PKEY/KV). ALL SYSTEMS USE QWEN2.5:7B-INSTRUCT WITH THE SAME PROMPT; ONLY THE CONTEXT DIFFERS. BEST PER COLUMN IN BOLD.

| System | NQA | Qasp | MFQA | HQA | 2WQA | MuSQ | GovR | QMS | PRet | PKey | KV | EnQA | **Avg** |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Full context | **26.8** | 51.4 | 51.7 | **65.6** | **58.7** | 24.2 | **15.9** | 19.4 | **93.3** | 16.7 | 10.0 | 23.0 | 38.1 |
| BM25 RAG | 18.7 | 45.8 | 47.5 | 51.4 | 38.0 | 17.6 | 11.7 | **20.6** | 70.0 | **100.0** | **96.7** | **34.3** | **46.0** |
| Dense RAG | 14.7 | 38.8 | 51.4 | 47.2 | 38.3 | 27.4 | 11.7 | 20.0 | 50.0 | **100.0** | 6.7 | 23.5 | 35.8 |
| SARA-lite | 17.2 | **53.5** | **55.0** | 61.4 | 51.9 | **33.5** | 12.0 | 18.8 | 16.7 | **100.0** | 16.7 | 22.0 | 38.2 |
| AMBER-lite | 11.1 | 42.6 | 53.3 | 44.8 | 43.1 | 32.9 | 9.9 | 18.9 | 0.0 | **100.0** | 10.0 | 18.0 | 32.0 |
| SketchRAG-sketch | 13.2 | 24.1 | 36.5 | 39.9 | 28.3 | 18.6 | 11.8 | 17.5 | 46.7 | **100.0** | 6.7 | 16.0 | 29.9 |
| SketchRAG-raw | 20.3 | 38.6 | 50.7 | 52.8 | 28.5 | 18.8 | 13.2 | 19.4 | 53.3 | **100.0** | **96.7** | 17.5 | 42.5 |
| SketchRAG-gate | 15.0 | 36.2 | 44.0 | 50.2 | 33.3 | 21.9 | 11.7 | 18.9 | 53.3 | **100.0** | **96.7** | 19.0 | 41.7 |
| SketchRAG-agent | 8.9 | 20.5 | 36.0 | 32.8 | 20.6 | 12.3 | 12.3 | 18.1 | 30.0 | **100.0** | 60.0 | 10.0 | 30.1 |
| SketchRAG-hybrid | 14.3 | 33.1 | 41.8 | 47.5 | 22.6 | 17.9 | 12.4 | 18.7 | 40.0 | **100.0** | 93.3 | 17.6 | 38.3 |

TABLE III

EFFICIENCY PER QUESTION (MEANS OVER 350 QUESTIONS). TOK = LLM CONTEXT TOKENS; REC = ANSWER RECALL OF THE CONTEXT (GOLD STRING PRESENT); FB = GATE FALLBACK RATE.

| System | Tok | Calls | LLM s | Rec | Fb |
|---|---|---|---|---|---|
| Full context | 14,232 | 1.00 | 61.5 | 0.51 | 0.00 |
| BM25 RAG | 1,327 | 1.00 | 8.1 | 0.49 | 0.00 |
| Dense RAG | 1,361 | 1.00 | 6.9 | 0.41 | 0.00 |
| SARA-lite | 2,435 | 1.00 | 12.9 | 0.45 | 0.00 |
| AMBER-lite | 2,223 | 2.97 | 31.8 | 0.33 | 0.00 |
| SketchRAG-sketch | 1,284 | 1.00 | 6.1 | 0.35 | 0.00 |
| SketchRAG-raw | 1,549 | 1.00 | 7.6 | 0.50 | 1.00 |
| SketchRAG-gate | 1,213 | 1.00 | 5.8 | 0.46 | 0.54 |
| SketchRAG-agent | 2,727 | 1.16 | 9.4 | 0.40 | 0.06 |
| SketchRAG-hybrid | 1,553 | 1.02 | 6.3 | 0.46 | 0.55 |

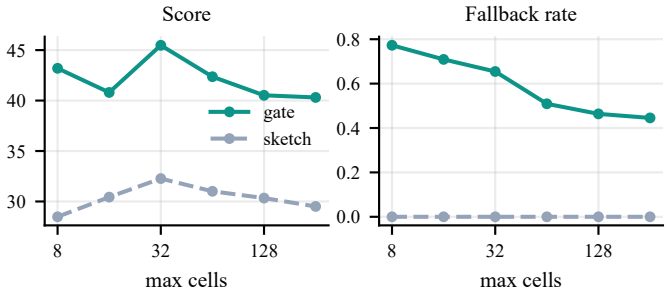



Fig. 4. Cell budget sweep (110 questions). The gate holds the score by escalating more often as the budget shrinks.

Leaving the decision to the LLM does not work at this scale: the AGENT policy called a tool on only 30 of 350 questions, mostly KV, and scored 30.1, close to digests alone. HYBRID (38.3) adds tokens and scores below the gate alone, most likely because extra tool output distracts the 7B model. The calibrated signal therefore comes from the sketch's error bound, not from the model's self-assessment.

### C. Ablations and memory budget (RQ3, RQ5)

Table V varies one component at a time on 110 questions. Raw pointers matter most: capping each cell at a 16-pointer reservoir [11] drops the score from 42.4 to 28.9, the digest-only level, because the chunk holding the answer is often no longer reachable. Removing the temporal neighbour costs 1.7 points; the other variants are within run-to-run noise (rerunning the frozen configuration gives 42.4 and 43.3).

Fig. 4 varies the cell budget from 8 to 256. Fewer cells give coarser digests: digest-only answer recall falls from 0.37 to 0.28, and the gate compensates by escalating more often (fallback 0.45 to 0.77), so the gated score stays between 40.3 and 45.5 with no monotone trend while hot memory shrinks from 507 to 178 KB. Narrowing the Count-Min Sketch loosens its $\varepsilon N$ bound, so the gate trusts fewer cells (fallback 0.51 at width 512, 0.75 at 64) while the score stays within 40.0–43.3; width 2048 more than doubles hot memory (1,023 vs. 416 KB) without lowering the fallback rate. We therefore keep 64 cells and width 512. Note that on a single LongBench document a 64-cell memory with exemplars is about as large as a dense index (454 vs. 447 KB); the bound pays off on long streams.

### D. Long-stream stress test (RQ3, RQ5)

On the 8.5M-word stream (Fig. 5), hot memory stays at 1.0 MB with 64 cells and 3.7 MB with 256, while a dense index grows to about 146 MB (raw text plus one 384-d vector per chunk; the raw text alone is 50 MB). Ingest costs 4.5 ms per chunk on a CPU. Recall, however, drops: SketchRAG-gate finds the evidence for 0.23 of probe questions (256 cells) against 0.44 for dense RAG and 0.38 for BM25. Every probe escalates (fallback 1.00), so SketchRAG behaves like a two-level router over cold storage: with thousands of unrelated documents sharing a few hundred cells, the cell centroid is too blunt a key.

### E. Comparison with EM-LLM (RQ2)

We ran EM-LLM's official code on two T4 GPUs. The hardware forced changes we record in our repository: weights in fp16 (no bf16 on T4), a smaller GPU block cache, a model split over both GPUs, and disk offload. Its $\infty$-Bench and NarrativeQA runs exceeded the 29 GB host-RAM limit, so it completed 6 of the 9 planned tasks (120 questions). On those, with Phi-3.5-mini as the shared model, SketchRAG-gate and EM-LLM are within a point (34.8 vs. 34.5, Table VI), with full context highest (37.7). SketchRAG needs one generation per question rather than streaming the whole document through

TABLE IV

REAL ANSWERS FROM THE MAIN RUN (QWEN2.5:7B). THE GATE'S ESCALATION REASON IS SHOWN PER QUESTION. THE LAST ROW IS A FAILURE: THE GATE ESCALATED, BUT THE EVIDENCE WAS NOT IN THE RETRIEVED CELLS.

| Task | Question (abridged) | Gold | Digest only | SketchRAG-gate | Full context | Gate |
|---|---|---|---|---|---|---|
| NQA | Who did Baron Conrad kill? | Baron Frederick | not enough informa… | Baron Frederick | Baron Henry | unresolved |
| KV | Value for key 35b9fdc1-c2c2-…? | e2e6376e-5c57-4… | 14f1 | e2e6376e-5c57-4b42… | b7000000-202d-4b0b… | unresolved |
| MuSQ | What is the place of birth of who does the voice of stan on the series that Insecurity is… | Denver | Malcolm McDowell | Denver | Trey Parker | lossy |
| 2WQA | Who is younger, Peter Rosegger or Ruel Redinger? | Ruel Redinger | Peter Rosegger | Peter Rosegger | Ruel Redinger | unresolved |

TABLE V

ABLATIONS OF SKETCHRAG-GATE (110 QUESTIONS, 11 TASKS × 10). Δ IS RELATIVE TO THE FROZEN CONFIGURATION.

| Variant | Score | Δ | Fb |
|---|---|---|---|
| Frozen configuration | 42.4 | – | 0.51 |
| 16-pointer reservoir per cell | 28.9 | -13.5 | 0.51 |
| Noise-floor confidence | 42.7 | +0.3 | 0.61 |
| No temporal neighbour | 40.7 | -1.7 | 0.48 |
| No co-occurrence check | 42.5 | +0.1 | 0.49 |
| Raw added, digest kept | 41.5 | -0.9 | 0.51 |
| CMS evidence in ranking ($w_{lex}$=0.5) | 41.7 | -0.7 | 0.54 |

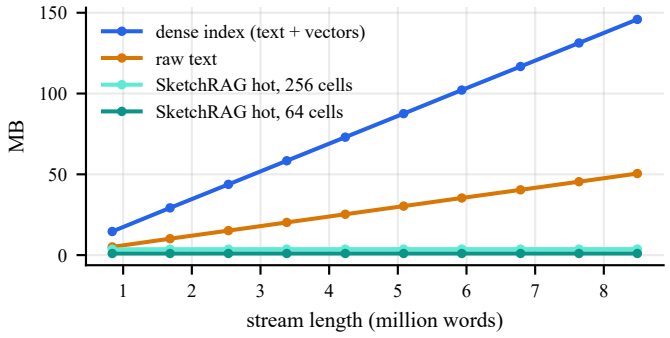



Fig. 5. Searchable memory vs. stream length on the 8.5M-word stress stream. SketchRAG's hot memory is flat; cold storage (not shown) grows like the raw text.

the model. EM-LLM ran in fp16 while our Phi-3.5 is Ollama's quantised build, so the comparison is on the same questions, not an identical model.

## VIII. DISCUSSION

**RQ1.** Extractive streaming summaries preserve topical content but lose exact facts; the raw text they point to is what recovers them. **RQ2.** With the gate, SketchRAG beats full context, dense RAG, SARA-lite and AMBER-lite at the lowest context cost and matches EM-LLM on a shared model, but BM25 remains stronger and multi-hop questions remain weak. **RQ3.** Within a document, the budget can shrink to 8–32 cells without losing accuracy because the gate compensates; across an 8.5M-word stream, memory stays flat but recall falls. **RQ4.** A deterministic error-bound gate decides well when the summary is insufficient; a 7B model asked to decide for itself does not. **RQ5.** The frozen configuration (64 cells, width 512, two raw chunks per escalated cell) is the best balance we found; raw pointers must not be cut.

TABLE VI

MATCHED COMPARISON WITH EM-LLM'S OFFICIAL CODE: SAME BASE MODEL (PHI-3.5-MINI) AND THE SAME 120 QUESTIONS ON THE 6 LONGBENCH TASKS EM-LLM COMPLETED.

| System | Qasp | MFQA | HQA | 2WQA | MuSQ | PRet | Avg |
|---|---|---|---|---|---|---|---|
| Full context | 39.7 | 41.6 | 38.3 | 35.0 | 11.5 | 60.0 | 37.7 |
| SketchRAG-raw | 34.1 | 40.3 | 56.4 | 18.6 | 24.5 | 43.5 | 36.2 |
| SketchRAG-gate | 40.3 | 47.4 | 47.3 | 16.2 | 20.0 | 37.5 | 34.8 |
| BM25 RAG | 31.4 | 55.0 | 42.4 | 12.1 | 26.7 | 40.0 | 34.6 |
| EM-LLM (official) | 43.2 | 50.9 | 32.9 | 27.7 | 37.5 | 15.0 | 34.5 |
| Dense RAG | 37.5 | 45.0 | 46.3 | 29.8 | 27.7 | 17.5 | 34.0 |

**Limitations.** The hot memory is bounded but cold storage is not; only the SKETCH policy is bounded end to end. SARA-lite and AMBER-lite are re-implementations on a shared LLM, not reproductions, and EM-LLM ran at reduced scale. Answer recall is a string-containment proxy rather than recall against annotated evidence. We use one 7B model and 30 questions per task, so differences under about two points are within noise.

**Live demonstration.** We also built a web console (Fig. 6) in which an agent monitors a 400k-word feed of LongBench documents, its 256-cell memory updating live, and answers typed questions with a local 3B model (llama3.2:3b through Ollama on a laptop CPU) while showing each retrieval step, the gate's verdict per cell and the raw evidence it read.

On that feed the gated context holds the answer for 27% of 56 questions (dense retrieval 25%), consistent with the stress-test finding.

## IX. CONCLUSION

SketchRAG shows that classic streaming synopses can serve as a bounded retrieval memory for an LLM agent, provided the system knows when not to trust them. The Count-Min error bound supplies that knowledge: it escalates exactly the questions where raw evidence matters and leaves the rest to compact digests, giving competitive accuracy at the lowest context cost of the systems we tested. The main open problem is recall at stream scale, where a few hundred centroids are too coarse a key; finer-grained keys, such as a small lexical index over digests or hierarchical cells, and a gate aware of multi-hop questions are the natural next steps. Code, results and the demo are in our project repository.

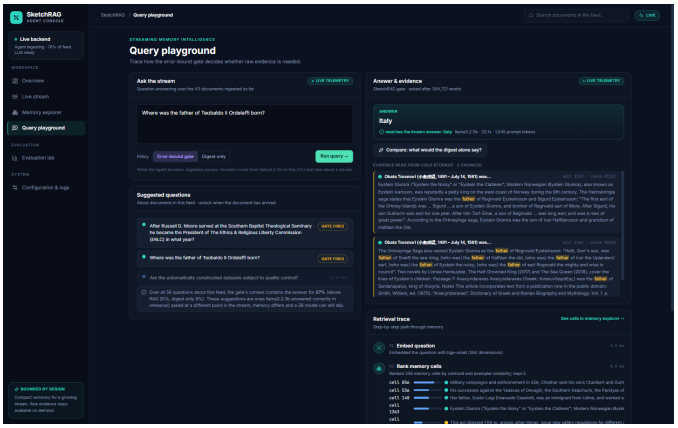



Fig. 6. Live demo console: a question asked mid-stream, the answer streamed by a local model and checked against the known answer, the raw chunks read from cold storage, and the retrieval trace (bottom).

## REFERENCES


[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] Z. Fountas, M. A. Benfeghoul, A. Oomerjee, F. Christopoulou, G. Lampouras, H. Bou Ammar, and J. Wang, "Human-inspired episodic memory for infinite context LLMs," in *International Conference on Learning Representations (ICLR)*, 2025.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[4] S. Muthukrishnan, "Data streams: Algorithms and applications," *Foundations and Trends in Theoretical Computer Science*, vol. 1, no. 2, pp. 117–236, 2005.

[5] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The Count-Min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[6] J. Misra and D. Gries, "Finding repeated elements," *Science of Computer Programming*, vol. 2, no. 2, pp. 143–152, 1982.

[7] Y. Jin, K. Sharma, V. Rakesh, Y. Dou, M. Pan, M. Das, and S. Kumar, "SARA: Selective and adaptive retrieval-augmented generation with context compression," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.

[8] Q. Qin, Y. Luo, Y. Lu, Z. Chu, X. Liu, and X. Meng, "Towards adaptive memory-based optimization for enhanced retrieval-augmented generation," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[9] "Ollama," https://ollama.com, accessed October 2026.

[10] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023.

[11] J. S. Vitter, "Random sampling with a reservoir," *ACM Transactions on Mathematical Software*, vol. 11, no. 1, pp. 37–57, 1985.

[12] T. Zhang, R. Ramakrishnan, and M. Livny, "BIRCH: An efficient data clustering method for very large databases," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1996, pp. 103–114.

[13] S. Guha, A. Meyerson, N. Mishra, R. Motwani, and L. O'Callaghan, "Clustering data streams: Theory and practice," *IEEE Transactions on Knowledge and Data Engineering*, vol. 15, no. 3, pp. 515–528, 2003.

[14] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," 2023.

[15] J. H. Ward, "Hierarchical grouping to optimize an objective function," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.

[16] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[17] X. Zhang, Y. Chen, S. Hu, Z. Xu, J. Chen, M. K. Hao, X. Han, Z. L. Thai, S. Wang, Z. Liu, and M. Sun, "$\infty$Bench: Extending long context evaluation beyond 100K tokens," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[18] Qwen Team, "Qwen2.5 technical report," 2024.

[19] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[20] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[21] M. Abdin *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," 2024.